A1.1 — Map processing and path planning (A*)

The `map_planner_node` loads the 3D point cloud of the hangar, converts it into a voxel grid, fills the racks as solid boxes, inflates every obstacle by a safety radius and searches this map for a route with 3D A*. The route is then simplified to a few straight segments and checked once more against the inflated map. The A* itself lives in `a_star_planner.cpp` and does not depend on ROS, so it is easy to test.

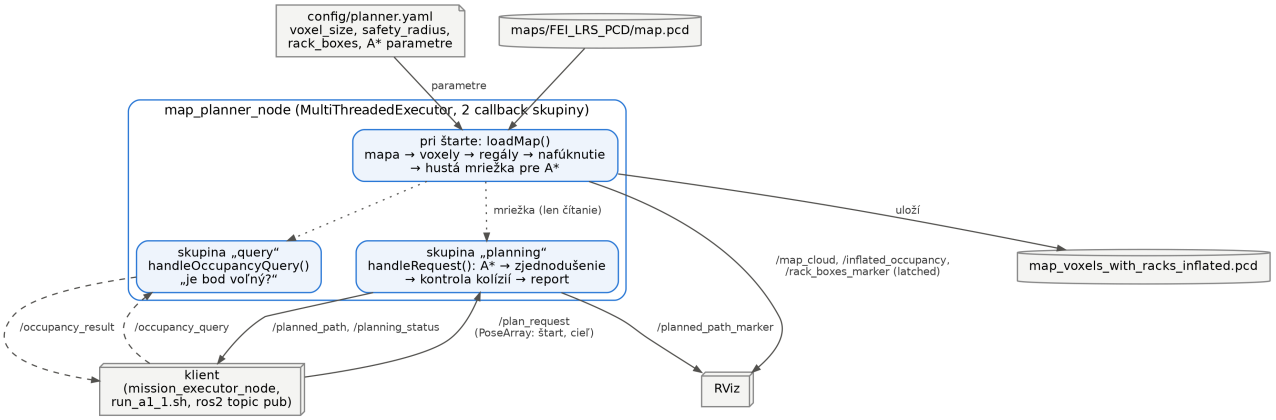
Run: `./run_a1_1.sh` (builds the package, starts the planner and RViz and requests a demo route). Other start and goal: `./run_a1_1.sh --start 12 4 2 --goal 16 8 2`, other radius: `--safety-radius 0.75`. More routes can be sent to `/plan_request` without a restart. **Measurements:** `python3 tools/a1_1_benchmark.py` (no simulator) saves every number in this document to `docs/A1_1/data/`.

Diagrams

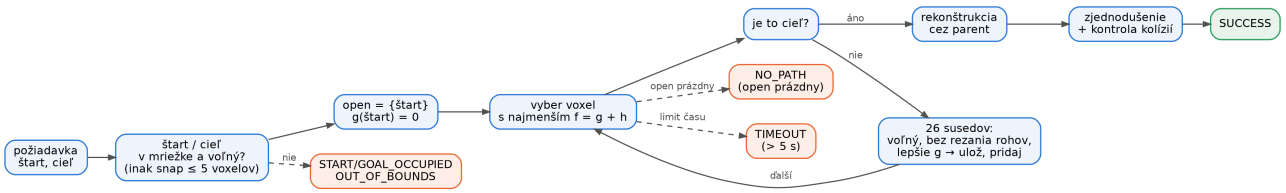
Data flow: the map is processed once at start-up; each request then only searches the finished grid.



ROS diagram: one node, two callback groups, so a long planning request never blocks an occupancy query. The grid does not change after start-up, so both threads only read it and no mutex is needed.



Algorithm: A* over a dense 3D grid. Every branch ends in a status that the planner also prints (dashed arrows are failure cases).



Decisions

Map loading

What: the map `maps/FEI_LRS_PCD/map.pcd` (620 311 points) is loaded with PCL (`pcl::io::loadPCDFile`) and its bounds are printed: x = [−0.30, 17.95] m, y = [−1.15, 13.20] m, z = [0.00, 5.95] m. The map path is

a ROS parameter (relative to the installed package, or absolute).

Why: PCL loads PCD files and downsamples them (VoxelGrid), and the result goes straight to RViz. A custom parser would add nothing.

Pros / cons: + proven library, fast (the whole planner start-up takes ~1.5 s). – One more dependency (`libpcl-dev`).

Voxel grid, 0.15 m

What: the cloud is downsampled with `VoxelGrid` to 0.15 m (62 689 points) and the occupied voxels are stored in a sparse hash set. Each voxel is one 64-bit key (three 21-bit indices). "Free" means "not in the set". For A*, occupancy is copied into a dense array of 128 × 102 × 46 voxels, where a query is a single array lookup.

Why: planning over raw points would not finish in reasonable time. 0.15 m is a compromise: finer makes A* expand many more voxels, coarser closes narrow passages (the drone radius is ~0.33 m). The grid is **fully 3D**, not stacked 2D layers: A* also moves diagonally up and down, for example over a rack.

Pros / cons: + "is this point free?" in O(1), + the sparse set saves memory (the hangar is mostly empty). – A voxel rounds positions: a real obstacle can be up to ~0.075 m closer than the voxel centre suggests (see results).

Any point can be queried from outside: publish to `/occupancy_query` (PointStamped) and the node answers on `/occupancy_result` (`true` = occupied or outside the grid). For example the rack centre (4.36, 6.82, 2.80) returns `true`.

Racks: solid boxes

What: both racks are filled as solid boxes (parameter `rack_boxes` in `planner.yaml`: centre, size, yaw). The sizes come from the collision shapes in the Gazebo world (`models/fei_lrs_racks/model.sdf`): uprights 3.76 m high, footprint 6.50 × 1.05 m, centres (4.32, 6.82) and (4.32, 11.36). Filling added 5 419 and 5 893 voxels.

Why: the cloud only sees the shelf surfaces. Between the shelf levels there are "free" voxels and A* would happily route through a shelf. The assignment quotes a 6.5 × 3.76 × 5.59 m collision box, but in the current world file the rack is built from shelves and uprights with a total height of 3.76 m, so we used the real world dimensions. Flying **over** a rack (z ≥ ~4.1 m) is therefore valid.

Pros / cons: + a rack is truly impassable, + sizes are parameters (no numbers in code). – If a rack moves in the world, the YAML must be updated; the node refuses to start without racks (`require_rack_boxes: true`).

Obstacle inflation (safety_radius 0.45 m)

What: every occupied voxel grows into a sphere of radius `safety_radius` (ROS parameter in `config/planner.yaml`, now 0.45 m), in 3D, so also above and below. The drone can then be treated as a point: every free voxel is safe for its centre.

Why 0.45 m: drone radius to the propeller tips 0.33 m (body 0.35 × 0.35 m, propellers at ±0.15 m) + ~0.12 m for position tracking error. We verified the value by flying the same mission in A1.2 (`results/A1_2/`): with 0.30 m (less than the drone itself) the drone sank to **0.20 m** from the rack edge on the

descent from the rack, so the propellers would hit it; with 0.45 m the smallest clearance was **0.37 m**. A larger radius visibly changes the route (figure below).

Pros / cons: + simple and safe also above and below obstacles. – Too large a radius closes narrow passages. – Mission points closer than ~0.45 m to a wall are blocked (handled by "snap", see below).

A* and the heuristic

What: A* over the dense grid, 26 neighbours (2D and 3D diagonals), step cost = step length (1, √2 or √3 voxels). The heuristic is the **3D octile distance**: sort the axis distances d1 ≤ d2 ≤ d3 and

```
h = voxel · [ (√3 − √2)·d1 + (√2 − 1)·d2 + d3 ]
```

Why: it is the exact length of the shortest path in empty space with 26 neighbours. Obstacles can only make the path longer, so h never overestimates (**admissible**). It is also **consistent**, so no voxel is expanded twice and the result is the shortest route on the grid. Euclidean distance would also be admissible but smaller, so A* would expand more voxels. Manhattan distance would overestimate on diagonals.

Pros / cons: + optimal route in milliseconds (4–46 ms). – Optimal only on the grid, not in continuous space (simplification makes up for part of that).

No corner cutting

What: a diagonal step is allowed only if **all** voxels of the cube it crosses are free (2 × 2 or 2 × 2 × 2).

Why: otherwise the route could slip between two occupied voxels that touch only at an edge or a corner. No drone fits through such a gap.

When no route exists

What: the planner always terminates and says why: `START/GOAL_OUT_OF_BOUNDS`, `START/GOAL_OCCUPIED`, `NO_PATH` (searched the whole reachable space) or `TIMEOUT` (5 s limit). The message on `/planning_status` also includes the reason and a hint (nearest free point, how close A* got to the goal).

Why: every voxel is closed at most once, so the open list eventually empties. The time limit protects against large maps. The A1.2 executor stops the mission safely based on this message.

Pros / cons: + never hangs. – `NO_PATH` into the closed office takes ~0.6 s, because the whole hangar has to be searched (331 458 voxels).

Moving the start and goal (snap)

What: if the start or goal lies in the inflated space, the nearest free voxel within 5 voxels (0.75 m) is used. In A1.2 the executor rejects the route if the goal moved by more than the waypoint tolerance (0.15 m hard, 0.55 m soft).

Why: a mission point 0.4 m from a wall is still safe for the drone's centre but lies in the inflated zone. Without snapping it could not be planned at all.

Path simplification

What: line-of-sight shortcutting. From point i jump to the farthest later point j that can be reached by a free straight segment, and continue from j. Each segment is sampled every quarter voxel (0.0375 m) against the inflated map.

Why: the raw A* route is a staircase of voxels (31–104 points). In A1.2 every point would mean slowing down, so the drone would move in hops. After simplification 2–5 points remain and the route is also shorter (e.g. 19.45 → 18.18 m).

Pros / cons: + far fewer points and a shorter route. – Sharp corners remain (no spline); the A1.2 executor turns on the spot before a sharp corner.

Collision check after simplification

What: before publishing, the node checks every point and every segment of the final route against the inflated map again. A colliding route is never published; an error is reported.

Why: shortcutting is exactly the step that can cut a corner through a shelf. A second, independent check catches that.

Results (no simulator)

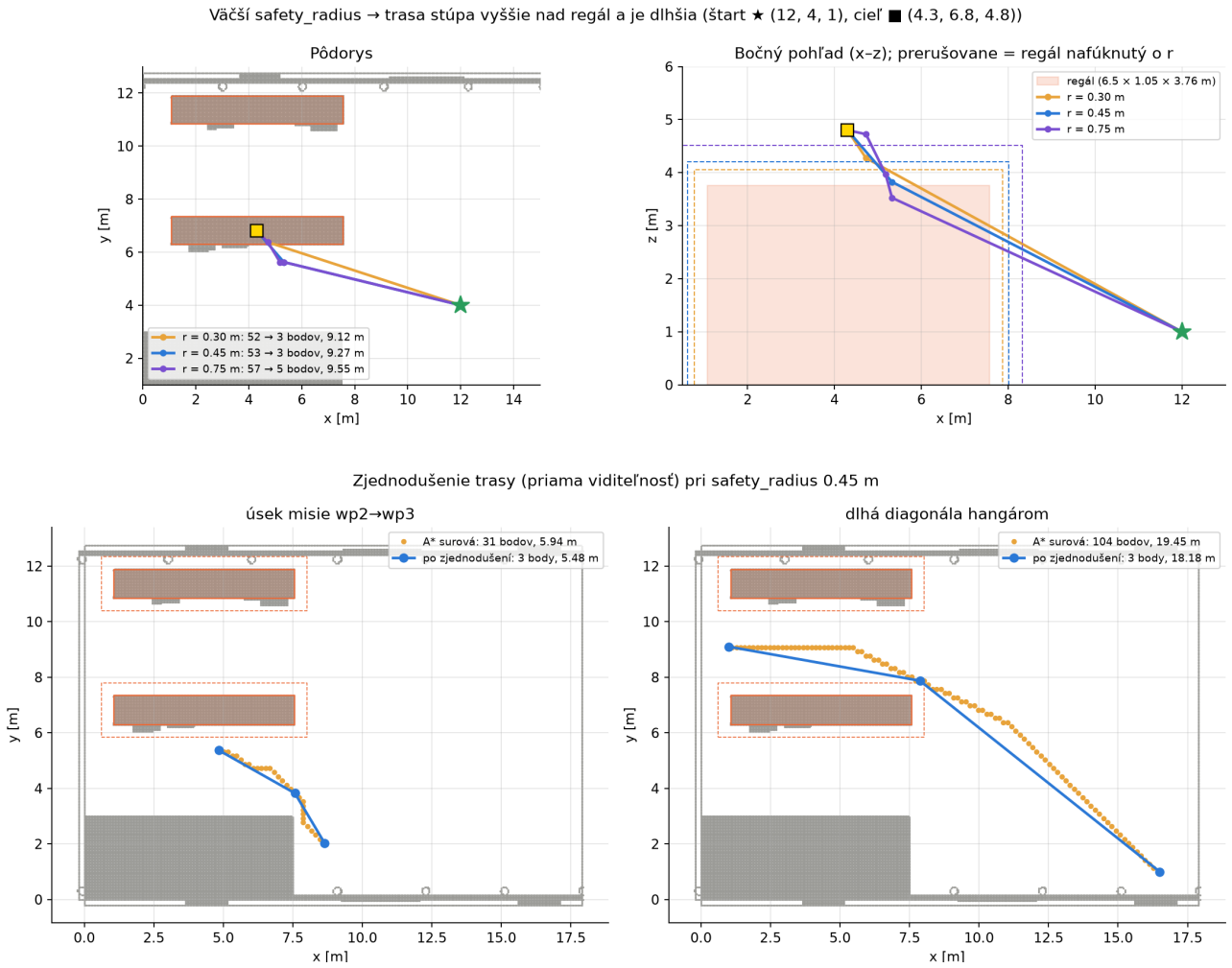
All numbers were measured by `tools/a1_1_benchmark.py` and saved in `docs/A1_1/data/benchmark.json` (raw routes in `benchmark_raw.json`). Map: 73 069 occupied voxels (with racks), 261 759 after inflation by 0.45 m.

Route (start → goal, m)	A* points → simplified	Length A* → final	Planning time
around a rack into the aisle (13, 7, 1.5) → (4.5, 9.1, 1.5)	58 → 2	9.42 → 8.76 m	4.4 ms
over a rack (12, 4, 1) → (4.3, 6.8, 4.8)	53 → 3	10.30 → 9.27 m	46.4 ms
mission leg (8.65, 2.02, 1) → (4.84, 5.37, 2)	31 → 3	5.94 → 5.48 m	14.7 ms
mission leg (2.08, 9.74, 1.75) → (8.84, 6.9, 2)	48 → 3	8.12 → 7.49 m	26.2 ms
long diagonal (16.5, 1, 1) → (1, 9.1, 3)	104 → 3	19.45 → 18.18 m	8.3 ms
goal in the middle of a rack	`GOAL_OCCUPIED`	—	0.0 ms
goal outside the map (25, 7, 1.5)	`GOAL_OUT_OF_BOUNDS`	—	0.0 ms
goal in the closed office (3, 1.5, 0.8)	`NO_PATH`, 331 458 voxels	—	618 ms

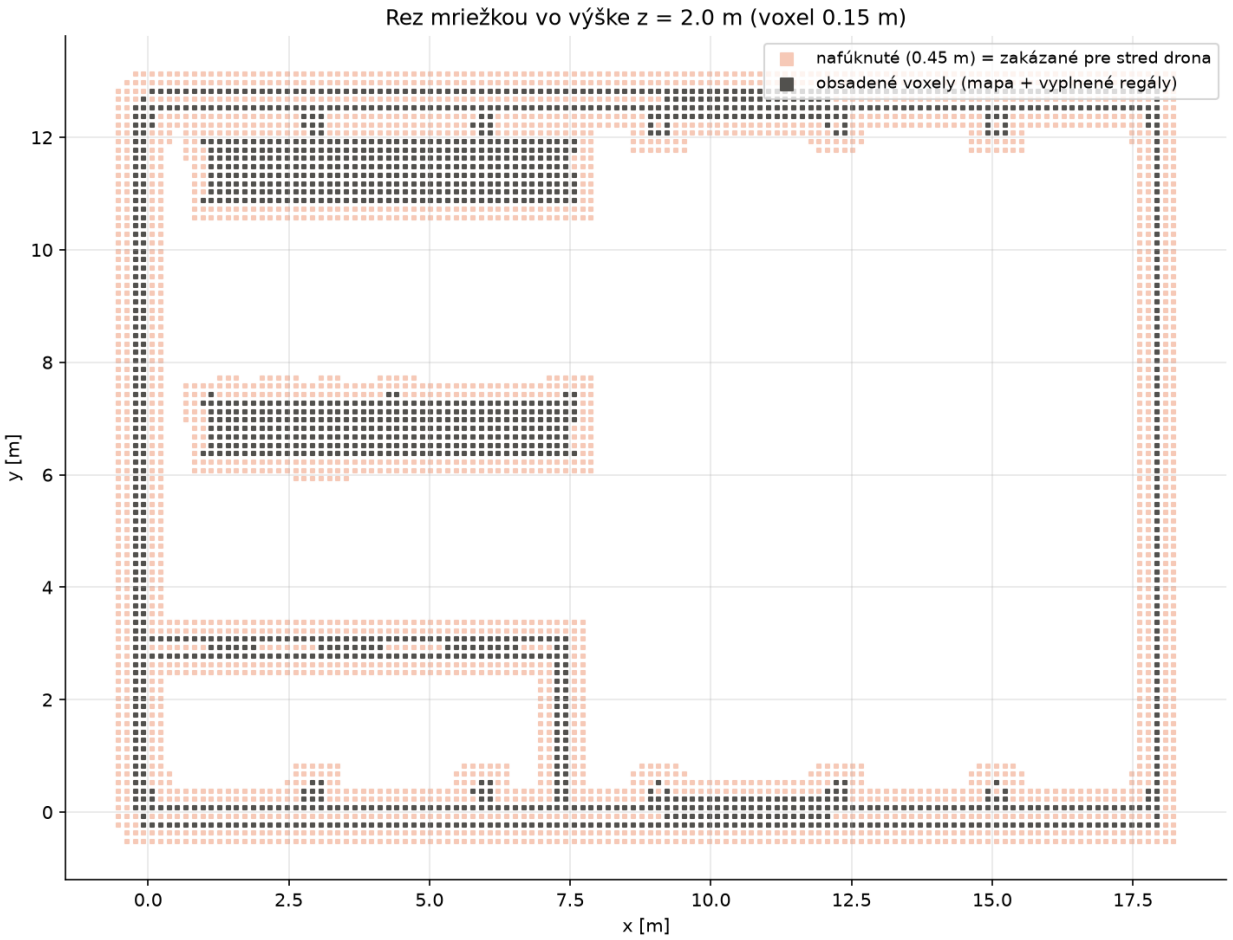
All successful routes passed the independent collision check. The planner also produced every route flown in A1.2 (three missions, 22 legs); at 0.45 m every flight kept more than the drone radius.

Effect of safety_radius on the route over a rack:

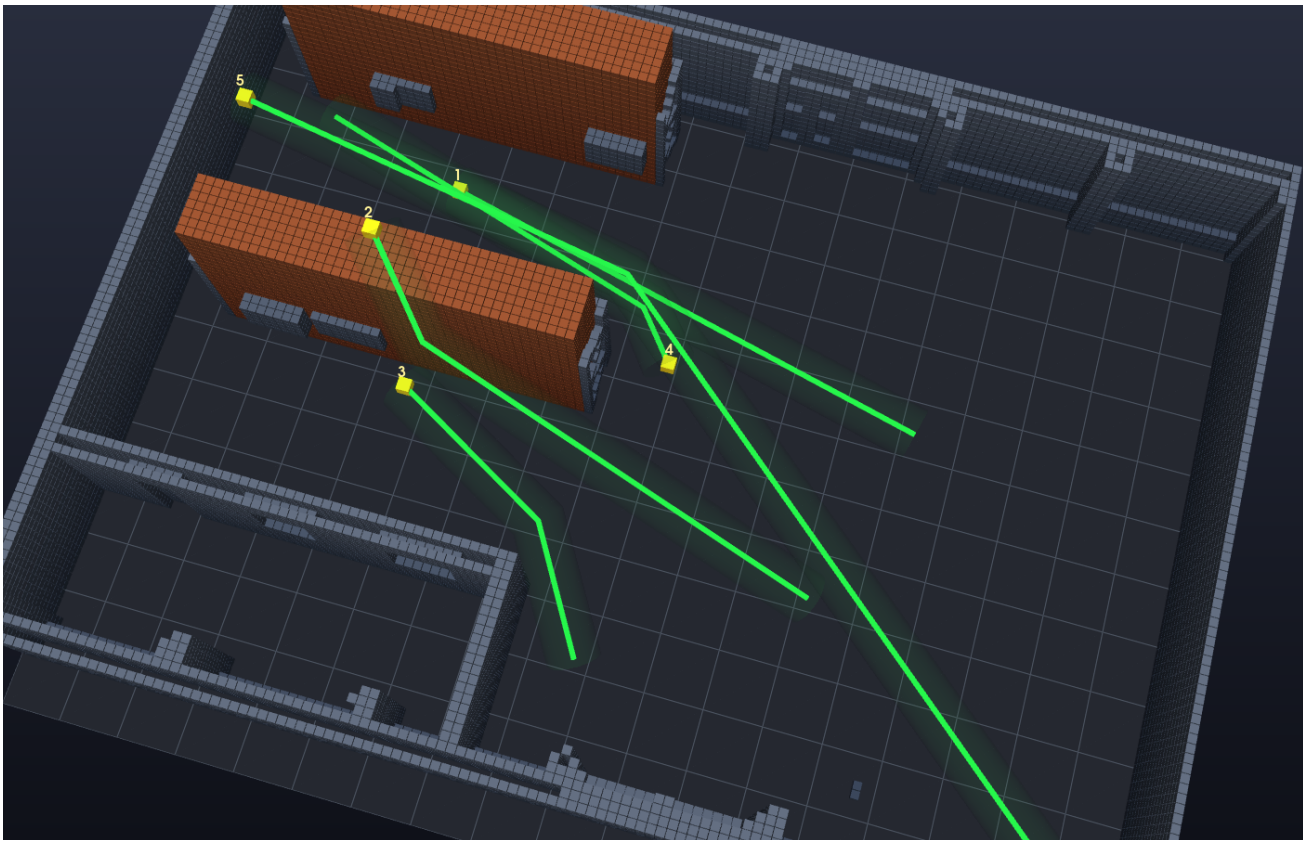
safety_radius	Points	Length	Time
0.30 m	52 → 3	9.12 m	24.2 ms
0.45 m (used)	53 → 3	9.27 m	46.4 ms
0.75 m	57 → 5	9.55 m	63.7 ms



Grid slice at 2 m: dark = occupied voxels (including the filled racks), light = inflated zone. It shows why the goal in the office at the bottom left is unreachable: the office is closed on all sides.



3D view of the tested routes (green tube = route, translucent hull = safety_radius, orange = racks, walls above 2.6 m hidden):



Known limitations

- **Holes in the roof of the cloud.** Along the roof ridge (y ≈ 4–7 m) the cloud has no points. A test with a goal above the roof (9, 6, 6.35) therefore **found a route out** through a hole. In Gazebo the roof has a collision model, so such a route must never be flown. Mission points are always inside the hangar, but the holes could be closed with another box in `rack_boxes` or with a ceiling (maximum height).
- **Voxel rounding:** a real obstacle can be up to ~0.075 m closer than the voxel centre. The route over the rack therefore had a real clearance of 0.37 m instead of 0.45 m (still more than the 0.33 m drone radius). A guaranteed 0.45 m needs `safety_radius` ≈ 0.53 m.
- **No smoothing (spline):** the route has sharp corners, where the drone in A1.2 briefly stops and turns.
- **NO_PATH is slow** (~0.6 s), because A* must search the whole reachable space.
- **Optimal only on the 0.15 m grid:** the simplified route is shorter, but not the shortest possible in continuous space.